Contents

Codex task: Frontend API utilisation and contextual map analytics **3**
Purpose . . . 3
Reviewed baseline . . . 3

1. First deliverable: complete endpoint-utilisation audit **3**
Baseline route inventory . . . 3
Currently used by the frontend . . . 3
Not currently used by the reviewed frontend . . . 4

2. Audit partial payload utilisation **4**
Sale transaction detail . . . 5
Required frontend use . . . 5
Rental transaction detail . . . 5
Building statistics . . . 5
Required frontend use . . . 6
Overview and time-series responses . . . 6

3. Target UX: hierarchical contextual statistics panel **6**
Desktop layout . . . 6
Mobile layout . . . 6
Context state model . . . 6
Hierarchy and breadcrumbs . . . 7
Zoom behavior . . . 7
URL state . . . 7

4. Geography context panel **7**
Header . . . 7
Primary metrics . . . 8
Trend and distribution . . . 8
Child comparison . . . 8

5. Building context panel **8**
Identity . . . 9
Market history . . . 9
Transactions . . . 9
Comparison . . . 9

6. Transaction/property contextual valuation **9**
Add a contextual-comparison endpoint . . . 9
Suggested response . . . 9
Comparison formula . . . 11
Comparable-cohort rules . . . 11
Nearby comparison . . . 11
Sample sufficiency . . . 11
Wording safeguards . . . 12

7. Use the existing valuation endpoint **12**

8. Use the existing market-index endpoint **12**

9. Outliers and anomaly signals **13**
Public transaction view . . . 13
Public market view . . . 13
Analyst/admin screen . . . 13

10. Authentication and preferences **13**

11. Frontend architecture requirements **14**
Decompose the map . . . 14
API client . . . 14
State and rendering . . . 14

12. Backend additions and performance **14**
Required additions . . . 14
Query constraints . . . 15
Performance targets . . . 15

13. Testing requirements **15**
Backend . . . 15
Frontend . . . 15
End-to-end scenarios . . . 16

14. Implementation phases **16**
Phase 0 — Audit and UX contract . . . 16
Phase 1 — Use existing backend data fully . . . 16
Phase 2 — Contextual comparison backend . . . 16
Phase 3 — Refactor map and finish interaction . . . 17
Phase 4 — Dedicated analyst and user features . . . 17

15. Required Codex report **17**

16. Definition of done **17**

17. Property detail rendering and missing-data policy **18**
Verified current behavior . . . 18
Required compact-popup behavior . . . 18
General rule . . . 18
Compact view versus detailed panel . . . 18
Compact map popup . . . 18
Detailed context panel . . . 19
Floor and position policy . . . 19

18. Audit additional real-estate attributes **19**
Already available in the sale transaction API but underused . . . 20
Useful fields stored in the imported source but not fully exposed by the API . . . 20
Building/property state . . . 20
Areas and outdoor space . . . 21
Address and identifiers . . . 21
Transaction composition . . . 21
Land . . . 21
Rental-specific attributes . . . 21
Recommended public property panel . . . 21
Core . . . 22
Optional when available . . . 22
Land-specific . . . 22
Rental-specific . . . 22
Derived values that may add real value . . . 22
Codebook descriptions . . . 23
Privacy and data-minimisation rules . . . 23

19. Display-value tests **23**

20. Additional definition of done **24**

Codex task: Frontend API utilisation and contextual map analytics

Purpose

The backend contains substantially more analytical functionality than the current frontend exposes. Audit all API operations, integrate valuable existing data, and redesign the map experience around a contextual statistics panel that follows the selected geography, building, or transaction.

This is a **separate task** from the general methodology and code-quality work. Do not mix it into unrelated refactoring commits.

Reviewed baseline

- Repository: `matkopolo/NepremicninskiAgent`
- Reviewed commit: `70b5ec82e964e2cf5061aa6855fb2a6b16bfe610`
- Backend route source: `apps/api/internal/httpapi/routes.go`
- Current frontend API client: `apps/web/lib/api.ts`
- Current frontend transport types: `apps/web/lib/types.ts`
- Main map component: `apps/web/components/dashboard/slovenia-map.tsx`

Re-audit the current branch before editing because the repository may have changed.

---

1. First deliverable: complete endpoint-utilisation audit

Create:

`docs/frontend-api-utilisation-audit.md`

For every registered API operation, record:

- method and path;
- backend handler/store;
- whether the frontend calls it;
- whether the frontend uses the complete response;
- intended audience:
  - public market dashboard;
  - authenticated user feature;
  - analyst/admin feature;
  - operational/health only;
- decision:
  - integrate now;
  - integrate in a dedicated screen;
  - intentionally keep backend-only;
  - extend before use;
  - remove only if genuinely obsolete;
- tests required.

Baseline route inventory

The reviewed backend registers 37 method/path operations. The current frontend directly uses approximately 17 of them.

Currently used by the frontend

1. `GET /api/v1/geographies`
2. `GET /api/v1/geographies/{id}/boundary`
3. `GET /api/v1/statistics/overview`
4. `GET /api/v1/statistics/timeseries`

5. `GET /api/v1/statistics/distribution`
6. `GET /api/v1/map/clusters`
7. `GET /api/v1/map/geographies`
8. `GET /api/v1/map/properties`
9. `GET /api/v1/rentals/statistics/overview`
10. `GET /api/v1/rentals/statistics/timeseries`
11. `GET /api/v1/rentals/statistics/distribution`
12. `GET /api/v1/rentals/map/clusters`
13. `GET /api/v1/rentals/map/geographies`
14. `GET /api/v1/rentals/map/properties`
15. `GET /api/v1/sales/transactions/{id}`
16. `GET /api/v1/buildings/search`
17. `GET /api/v1/buildings/{cadastralCode}/{buildingNumber}/statistics`

Verify this list against the current source rather than accepting it blindly.

Not currently used by the reviewed frontend

High-value public analytical operations

- `GET /api/v1/meta`
- `GET /api/v1/geographies/{id}`
- `GET /api/v1/geographies/{id}/children`
- `GET /api/v1/statistics/outliers`
- `GET /api/v1/rentals/statistics/outliers`
- `GET /api/v1/rentals/transactions/{id}`
- `GET /api/v1/indices`
- `GET /api/v1/valuations/{kind}/{id}`

Analyst/admin functionality

- `GET /api/v1/anomalies`
- `GET /api/v1/anomalies/summary`
- `PATCH /api/v1/anomalies/{kind}/{id}/review`

Authentication and saved-user state

- `POST /api/v1/auth/register`
- `POST /api/v1/auth/login`
- `POST /api/v1/auth/logout`
- `GET /api/v1/users/me`
- `GET /api/v1/users/me/preferences`
- `PUT /api/v1/users/me/preferences`

Operational endpoints

- `GET /health/live`
- `GET /health/ready`
- `GET /api/v1/health`

Operational endpoints do not need prominent dashboard UI. They may feed a subtle service-status diagnostic, monitoring system, or deployment health check.

---

2. Audit partial payload utilisation

An endpoint does not count as fully utilised merely because it is called.

Sale transaction detail

The backend already returns:

- dataset year;
- price derivation;
- coordinates;
- quality eligibility;
- representative eligibility;
- quality reasons;
- outlier status and method;
- outlier cohort level, size, median, bounds, and modified Z-score;
- full geography hierarchy;
- detailed building parts;
- detailed land parcels.

The reviewed frontend transport type and popup use only a subset.

Required frontend use

For a selected sale:

- show address and property identity;
- show contract date, area, amount, and amount per m^2;
- show whether the record is representative;
- explain exclusions or quality reasons;
- show outlier evidence as an analytical warning, not as a hidden backend detail;
- show geography breadcrumb;
- show price derivation;
- support land-parcel information where relevant;
- link to contextual valuation and comparison data.

Rental transaction detail

Implement `GET /api/v1/rentals/transactions/{id}` in the frontend.

Show:

- reported and VAT-exclusive rent;
- rent per m^2;
- rental start/end dates;
- duration and rental term;
- whether costs are included;
- whether it is classified as market rent;
- quality/outlier evidence;
- geography breadcrumb;
- rented-space details.

Building statistics

The backend already returns:

- sale summary;
- rental summary;
- sale history by year;
- rental history by year;
- first and last contract dates;
- representative sample counts;
- sale transactions.

The reviewed frontend type discards rentals, histories, representative counts, and date ranges.

Required frontend use

For a selected building:

- show sale and rental tabs;
- show yearly history;
- show representative count separately from raw transaction count;
- show first/last observed contract date;
- show all relevant transactions;
- show a low-sample warning;
- preserve the selected property group and year context.

Before presenting building comparisons, fix or extend the backend so apartment statistics are not mixed with garages, commercial units, or other property groups. Add `propertyGroup`, `from`, and `to` filters to the building endpoint, or create a dedicated contextual building endpoint.

Overview and time-series responses

Expose useful fields currently omitted from frontend types:

- eligible count;
- included sample size;
- excluded outlier count;
- raw versus cleaned averages;
- price/rent metric sample sizes;
- median area;
- incomplete-period flag.

Do not overwhelm the primary UI. Put methodology and sample-quality details in expandable sections or tooltips.

---

3. Target UX: hierarchical contextual statistics panel

Refactor the main dashboard into a map plus a persistent contextual panel.

Desktop layout

- map on the left;
- contextual statistics panel on the right;
- panel width approximately 360–460 px;
- charts below or inside expandable panel sections;
- the panel must not obscure the map.

Mobile layout

Use an accessible bottom sheet or full-width details section below the map.

Context state model

Introduce one explicit selection model instead of scattered map state:

```
type MarketContext =
  | { kind: "country"; geographyId: number }
  | { kind: "geography"; geographyId: number; level: GeographyLevel }
  | { kind: "building"; cadastralCode: number; buildingNumber: string }
  | { kind: "transaction"; transactionKind: "sale" | "rent"; transactionId: number }
  | { kind: "viewport"; bounds: [number, number, number, number]; level: GeographyLevel };
```

Names may differ, but there must be one source of truth shared by map, filters, URL state, and context panel.

Hierarchy and breadcrumbs

The expected drilldown is:

`Slovenija → Osrednjeslovenska → Ljubljana → Center → building → transaction`

Possible levels in the current data model include:

- country;
- statistical region;
- municipality;
- city district;
- local community;
- settlement;
- building;
- transaction/property.

Use `GET /api/v1/geographies/{id}` and `GET /api/v1/geographies/{id}/children` for named hierarchy navigation.

Zoom behavior

Zoom controls the semantic layer, but selection controls the statistical context.

Recommended behavior:

- broad zoom: show country/statistical-region summaries;
- medium zoom: show municipality summaries;
- closer zoom: show city district or local community summaries;
- street zoom: show buildings and individual properties;
- clicking a geography selects it and updates the panel;
- clicking a building selects the building;
- clicking a property selects the transaction;
- changing zoom alone may update a temporary viewport summary, but must not silently replace an explicit user selection.

If exactly one named geography clearly dominates the viewport, the UI may offer "Use Osrednjeslovenska as context." Do not make unstable automatic selections while the user is panning.

URL state

Persist at least:

- market mode;
- property group;
- year range;
- selected geography/building/transaction;
- map center and zoom where practical.

This makes a specific analytical view shareable and reload-safe.

---

4. Geography context panel

When a region such as **Osrednjeslovenska**, a municipality such as **Ljubljana**, or a district such as **Center** is selected, show statistics for the exact active filters.

Header

- geography name and level;
- breadcrumb;

- active property group;
- selected period;
- source freshness;
- raw and representative sample counts.

Primary metrics

For sales:

- median price per m²;
- median total price;
- cleaned average per m²;
- median area;
- representative transaction count;
- outlier-exclusion count.

For rentals:

- median monthly rent per m²;
- median monthly rent;
- median area;
- representative rental count;
- market-rent count where available;
- outlier-exclusion count.

Trend and distribution

Use existing endpoints:

- overview;
- time series;
- distribution.

Add:

- year-over-year change;
- incomplete-period warning;
- percentile/distribution summary;
- market index from `GET /api/v1/indices` where appropriate.

The index and raw median trend are different concepts. Label them clearly.

Child comparison

Use geography children to show a ranked list:

- municipalities within a statistical region;
- districts/local communities within a municipality;
- count, median per m², and change;
- click a child to zoom and select it.

Avoid N+1 requests. Prefer a backend batch summary or reuse `/map/geographies` response properties when they contain the required metrics.

---

5. Building context panel

When a building is selected, show:

Identity

- address;
- cadastral municipality and building number;
- construction year;
- known part count;
- full geography breadcrumb.

Market history

- sale history by year;
- rental history by year;
- representative/raw sample counts;
- first and last observed transaction;
- median amount per m^2.

Transactions

- sale and rental transaction lists;
- filters by year and property group;
- click a transaction to open transaction context.

Comparison

Compare the building against:

1. its city district/local community;
2. its municipality;
3. its statistical region;
4. Slovenia;
5. nearby comparable buildings, if a valid spatial cohort exists.

Do not compare a mixed building-wide median against an apartment without matching property group and period.

---

6. Transaction/property contextual valuation

The user's target example should become a first-class feature:

> "This apartment sold 25% above the nearby representative median and 30% above the Osrednjeslovenska median for apartments sold in the same year."

This requires a consistent backend-computed comparison response. Do not derive it by aggregating the limited set of map points currently loaded in the browser.

Add a contextual-comparison endpoint

Preferred resource shape:

- `GET /api/v1/sales/transactions/{id}/context`
- `GET /api/v1/rentals/transactions/{id}/context`

A unified alternative such as `GET /api/v1/transactions/{kind}/{id}/context` is acceptable if it matches the repository's route style and OpenAPI conventions.

Suggested response

```
{
  "transactionKind": "sale",
  "transactionId": 123,
```

```
"contractYear": 2025,
"propertyGroup": "apartment",
"actualAmountPerSqm": 4200,
"valuation": {
  "predictedAmountPerSqm": 3850,
  "lowerAmountPerSqm": 3300,
  "upperAmountPerSqm": 4400,
  "differenceFromPredictionPercent": 9.1,
  "reliability": "moderate",
  "modelVersion": "..."
},
"benchmarks": [
  {
    "scope": "nearby",
    "label": "Comparable sales within 1 km",
    "medianAmountPerSqm": 3360,
    "sampleSize": 24,
    "differencePercent": 25.0,
    "percentile": 83,
    "period": "2025",
    "radiusMeters": 1000,
    "reliability": "moderate"
  },
  {
    "scope": "city_district",
    "geographyId": 456,
    "label": "Center",
    "medianAmountPerSqm": 3500,
    "sampleSize": 105,
    "differencePercent": 20.0,
    "percentile": 78,
    "period": "2025",
    "reliability": "high"
  },
  {
    "scope": "statistical_region",
    "geographyId": 12,
    "label": "Osrednjeslovenska",
    "medianAmountPerSqm": 3231,
    "sampleSize": 2600,
    "differencePercent": 30.0,
    "percentile": 88,
    "period": "2025",
    "reliability": "high"
  },
  {
    "scope": "country",
    "label": "Slovenija",
    "medianAmountPerSqm": 2800,
    "sampleSize": 7600,
    "differencePercent": 50.0,
    "percentile": 93,
    "period": "2025",
    "reliability": "high"
  }
],
"quality": {
```

```
    "isRepresentative": true,
    "outlierStatus": "inlier",
    "warnings": []
  }
}
```

The exact schema may differ. Preserve these concepts.

Comparison formula

For a valid positive benchmark:

```
differencePercent =
  ((actualAmountPerSqm / benchmarkMedianAmountPerSqm) - 1) * 100
```

Round only for display. Preserve full precision in backend calculations.

Comparable-cohort rules

At minimum, comparisons must use:

- same transaction kind;
- same property group;
- the same calendar year as the selected transaction;
- representative-eligible records only;
- subject transaction excluded from its own cohort;
- correct land eligibility rules;
- no duplicate transaction inflation.

For land, the default verified-buildable comparison must include only strict ETN land codes 1 and 2. Code 3 must remain separately labelled.

Nearby comparison

Define "nearby" explicitly:

- PostGIS distance from the selected property;
- start with a documented radius such as 500 m or 1 km;
- expand through a deterministic fallback ladder only when sample size is too small;
- return the actual radius and sample size;
- never call a geography median "nearby" unless it is truly spatial.

Consider comparability filters such as area bands, building type, or age only after validating that enough observations remain. The model estimate provides attribute adjustment; the nearby median remains a transparent descriptive benchmark.

Sample sufficiency

Return sample size and reliability for every benchmark.

Suggested policy to validate and document:

- fewer than 5 observations: do not calculate a comparative statement;
- 5–9: show only with a prominent "very small sample" warning;
- 10–29: low/moderate confidence;
- 30 or more: stronger descriptive benchmark.

These thresholds are starting points, not unquestionable statistical truth. Codex must align them with the methodology guide and test sensitivity.

Wording safeguards

Allowed:

- “25% above the nearby representative median.”
- “30% above the Osrednjeslovenska apartment median in 2025.”
- “Actual price was 9% above the model estimate.”
- “The transaction is at the 83rd percentile of its comparison cohort.”

Avoid without stronger evidence:

- “25% overpriced.”
- “Bad purchase.”
- “The property is worth exactly €X.”
- “The model proves the seller overcharged.”

Raw cohort differences are descriptive. The valuation endpoint is an estimate with an interval and reliability label, not an official valuation.

---

7. Use the existing valuation endpoint

Integrate `GET /api/v1/valuations/{kind}/{id}` into selected transaction UI.

Show:

- actual amount per m^2;
- predicted amount per m^2;
- prediction interval;
- predicted total and interval;
- residual difference;
- model version/type;
- training-row count;
- reliability;
- all warnings.

Do not hide low or very-low reliability. If no prediction exists, show a neutral unavailable state.

The contextual-comparison endpoint may internally compose this data, but avoid duplicating valuation calculations in two backend paths.

---

8. Use the existing market-index endpoint

Integrate `GET /api/v1/indices` into a clearly labelled normalized trend view.

Possible UI:

- toggle: `Median €/m²` versus `Quality-adjusted index`;
- display base period and sample size;
- show method and methodology version;
- hide or warn for missing/weak periods.

The current backend index query is national-only. Do not label it as a selected district or municipality index. Add geography-level indices only after the methodology and sample size support them.

---

9. Outliers and anomaly signals

Public transaction view

Use transaction-level quality/outlier evidence to explain:

- why a transaction is excluded from representative statistics;
- whether it is a statistical outlier;
- its cohort median and bounds;
- the rule version.

Public market view

A subtle data-quality section may show:

- total records;
- representative sample;
- excluded outliers;
- incomplete periods.

Do not surface a raw anomaly-review queue in the public dashboard.

Analyst/admin screen

Create a separate authenticated screen for:

- anomaly summary;
- paginated anomaly list;
- filters by kind, flag, status, score;
- transaction drilldown;
- review action and notes;
- audit identity and timestamp.

Use:

- `GET /api/v1/anomalies`
- `GET /api/v1/anomalies/summary`
- `PATCH /api/v1/anomalies/{kind}/{id}/review`

This screen depends on authentication and should not block the public context panel.

---

10. Authentication and preferences

Treat auth/preferences as a separate product capability.

Useful preference examples:

- preferred market mode;
- default property group;
- default year range;
- favourite geographies;
- saved buildings;
- saved comparisons;
- map style;
- dismissed methodology explanations.

Do not implement auth merely to claim every endpoint is used. Integrate it only when the saved-user experience is coherent and CORS/auth correctness is fixed.

---

11. Frontend architecture requirements

Decompose the map

The current map component should not own all of:

- MapLibre lifecycle;
- semantic zoom;
- marker creation;
- popup DOM;
- network requests;
- selection state;
- building search;
- context-panel rendering.

Suggested modules:

- `use-market-context.ts`
- `use-maplibre-map.ts`
- `use-semantic-map-data.ts`
- `map-markers.ts`
- `map-popups.ts`
- `context-panel/context-panel.tsx`
- `context-panel/geography-context.tsx`
- `context-panel/building-context.tsx`
- `context-panel/transaction-context.tsx`
- `building-search-panel.tsx`

One marker rendering path must serve initial and refreshed data.

API client

- generate transport types from OpenAPI;
- use typed API errors;
- handle AbortError explicitly;
- cancel stale context requests;
- do not silently use demo data in production;
- cache stable metadata/geography data;
- avoid N+1 child/geography requests;
- keep UI view models separate from transport DTOs.

State and rendering

- show skeleton/loading state without clearing the previous valid context;
- mark stale data;
- prevent an older request from replacing a newer selection;
- preserve explicit selection while panning;
- make all comparisons accessible in text, not only chart colour.

---

12. Backend additions and performance

Before implementing new endpoints, inspect existing queries and indexes.

Required additions

Likely required:

1. transaction contextual-comparison endpoint;

2. property-group/date filters for building statistics;
3. optional batch child-geography summaries;
4. optional viewport statistics only if the UX truly needs non-geographic viewport summaries;
5. data-vintage metadata if `/meta` does not already provide it.

Query constraints

- use PostGIS spatial indexes for nearby cohorts;
- cap radius and result work;
- use representative analytical facts, not raw joined child rows;
- avoid repeated geography lookups;
- use one consistent transaction snapshot where multiple related values are returned;
- explain/analyze key queries;
- add indexes only with evidence;
- return sample counts and methodology versions;
- cache stable geography boundaries and metadata.

Performance targets

Set measured targets rather than assuming them. A reasonable initial goal for a local production-like database is:

- selected geography panel: p95 under 500 ms after warm-up;
- selected building panel: p95 under 700 ms;
- selected transaction context: p95 under 800 ms;
- map-pan data request: p95 under 500 ms.

Record dataset size, environment, query plan, and whether the result was cached. Adjust targets if the production architecture justifies different limits.

---

13. Testing requirements

Backend

Add tests for:

- route/OpenAPI contract;
- same-year filtering;
- geography hierarchy;
- nearby radius calculation;
- subject-transaction exclusion;
- property-group matching;
- representative-only cohorts;
- land codes 1–2 default;
- sample-size thresholds;
- percentage calculation;
- null/zero benchmark handling;
- incomplete year;
- missing valuation;
- low-reliability valuation;
- building filters;
- SQL integration on representative fixture data;
- query bounds and pagination.

Frontend

Add tests for:

- semantic zoom level;

- geography selection and breadcrumb;
- region → municipality → district drilldown;
- building selection;
- sale and rental transaction selection;
- stale-response cancellation;
- panel loading/error/empty states;
- exact comparison wording;
- positive, negative, and zero differences;
- sample-size warnings;
- valuation reliability/warnings;
- outlier explanation;
- URL persistence;
- keyboard navigation;
- responsive bottom-sheet behavior.

End-to-end scenarios

1. Open Slovenia and select apartments, 2025.
2. Select Osrednjeslovenska and verify panel filters/data.
3. Drill into Ljubljana.
4. Drill into Center.
5. Select a building and view matching apartment history.
6. Select a sale and see nearby, district, municipality, region, and national comparisons.
7. Verify the example statement is calculated from backend values.
8. Switch to rentals and open rental detail.
9. Simulate insufficient local sample and verify the warning/fallback.
10. Simulate API failure and verify no demo statistics masquerade as live data.

---

14. Implementation phases

Phase 0 — Audit and UX contract

- create endpoint-utilisation matrix;
- classify public/admin/operational routes;
- document context state and wireframes;
- define comparison schema and statistical rules;
- add characterization tests.

Phase 1 — Use existing backend data fully

- expand/generated frontend types;
- geography context panel;
- complete sale transaction detail;
- rental transaction detail;
- full building sale/rental history;
- meta/data freshness;
- market indices;
- valuation card;
- quality/outlier explanations.

Phase 2 — Contextual comparison backend

- add transaction-context endpoint;
- add building filters;
- add efficient hierarchical/nearby benchmarks;
- add OpenAPI and integration tests.

Phase 3 — Refactor map and finish interaction

- one selection model;
- one marker/popup path;
- context panel;
- URL state;
- cancellation and caching;
- mobile layout.

Phase 4 — Dedicated analyst and user features

- anomaly review screen;
- authentication;
- saved preferences/favourites.

Do not delay the public analytical experience for Phase 4.

---

15. Required Codex report

Create:

`docs/frontend-contextual-analytics-implementation-report.md`

Include:

- endpoint matrix before and after;
- intentionally unused endpoints and reasons;
- new/changed API contracts;
- comparison methodology;
- screenshots or component descriptions;
- query plans/performance measurements;
- tests run;
- known limitations;
- deferred work;
- final statement of whether the frontend now exposes the backend's most valuable analytical capabilities.

---

16. Definition of done

- Every backend operation is classified.
- Valuable public analytical endpoints are integrated or intentionally deferred with a documented reason.
- The frontend no longer discards quality, outlier, geography, rental-history, or valuation information needed for analysis.
- Region, municipality, district/local-community, building, and transaction contexts have distinct panel views.
- Clicking Osrednjeslovenska, Ljubljana, or Center displays statistics for that exact geography and active filters.
- A selected sale can display backend-computed differences versus nearby, municipality, statistical-region, and national representative cohorts for the same year/property group.
- The UI states sample size, comparison period, benchmark scope, and reliability.
- No client-side aggregation of the currently visible map subset is used as a market benchmark.
- No raw median comparison is labelled as proof that a property is overpriced.
- Rental transaction details work.
- Building statistics respect property group and period.
- Valuation intervals and warnings are visible.
- Map/context requests handle cancellation and stale responses.
- OpenAPI, backend tests, frontend tests, build, lint, and relevant integration tests pass.

17. Property detail rendering and missing-data policy

This section is mandatory. The frontend must not render a row merely because a label exists.

Verified current behavior

The reviewed sale popup currently builds a fixed row list and substitutes `"Ni podatka"` for missing values. In particular:

- `Nadstropje: Ni podatka` is generated by the frontend when `floor` is null, blank, or whitespace.
- `Položaj: nadstropje` is **not hardcoded**. It is the API value `positionInBuilding`, loaded from the database column `position_in_building`, which originates from the ETN source field `LEGA_DELA_STAVBE_V_STAVBI`.
- Although `nadstropje` is real source data, it is a generic position classification and is not the same as an actual floor number.

Required compact-popup behavior

The compact popup must show only concise, decision-useful values that are actually available.

For the example:

```
Uporabna površina: 29 m²
Nadstropje: Ni podatka
Položaj: nadstropje
```

the compact view should render only:

```
Uporabna površina: 29 m²
```

Do not show `Nadstropje` or `Položaj` in the compact popup for this case.

General rule

A field row must be omitted when its value is:

- `null` or `undefined`;
- an empty or whitespace-only string;
- `NaN` or an invalid number;
- a known source sentinel;
- a generic/non-informative codebook value that adds no useful information in the current UI context;
- logically contradictory to another displayed field.

Do not use `"Ni podatka"`, `"-"`, `"brez podatka"`, zero, or an empty string as a universal replacement inside the normal property summary.

An explicit missing-data message is acceptable only when the missing value itself is analytically important, for example:

- “Ocena ni na voljo, ker model nima dovolj primerljivih poslov.”
- “Za to območje ni dovolj reprezentativnih transakcij.”
- “Podatek o ceni ni javno razpoložljiv.”

Compact view versus detailed panel

Compact map popup

Limit it to approximately 4–7 useful items:

- property/address identity;
- contract date;
- total price or monthly rent;
- price/rent per m²;
- relevant area;
- one quality/reliability badge;

- action to open the full context panel.

Do not place every available ETN field in the popup.

Detailed context panel

The detailed side panel may show additional sections, but still omit absent rows rather than listing many “Ni podatka” values.

Suggested sections:

- Transaction
- Property
- Building
- Land
- Location
- Data quality
- Valuation and comparisons
- Methodology

Floor and position policy

- Remove `Nadstropje` and `Položaj` from the compact popup.
- In the detailed panel, show `Nadstropje` only when the source contains a meaningful actual floor value.
- Show `Lega v stavbi` only when it adds information beyond a generic value such as `nadstropje`.
- Normalize capitalization and labels through a codebook/domain translator.
- Never infer a floor number from the generic position value.
- Never display raw internal codes where a Slovenian description exists.
- Add a value-policy helper so this logic is not duplicated across sale, rental, building, and map components.

Suggested helper concept:

```ts
type DisplayField = {
  label: string;
  value: string | number | null | undefined;
  importance: "primary" | "secondary" | "technical";
  isMeaningful?: (value: unknown) => boolean;
};

function visibleFields(fields: DisplayField[]): DisplayField[] {
  // Trim, validate, remove absent/sentinel/non-informative values.
}
```

The exact API may differ. Keep the rule centralized and unit-tested.

---

18. Audit additional real-estate attributes

Create a field-availability matrix in:

`docs/frontend-property-field-audit.md`

For every useful source/database/API field, record:

- source ETN field;
- database column;
- codebook, unit, or semantics;
- whether it is exposed by the API;
- whether the frontend displays it;
- null rate or availability rate;
- whether it is reliable enough for public display;

- target UI location;
- privacy/sensitivity risk;
- final decision.

Do not add a field merely because it exists. Prioritize usefulness, reliability, interpretability, and sufficient population.

Already available in the sale transaction API but underused

The reviewed backend sale detail already contains useful fields including:

- property group;
- dataset year;
- area;
- price;
- price per m^2;
- price derivation;
- coordinates;
- quality eligibility;
- representative eligibility;
- quality reasons;
- outlier evidence;
- geography chain;
- building-part type;
- construction year;
- floor;
- position in building;
- unit number;
- room count;
- sold area;
- sold usable area;
- registered building-part area;
- usable area;
- outdoor parking-space count;
- contractual component price;
- settlement, street, and house number;
- land parcel type, area, number, and component price.

The frontend type currently exposes only part of this information. Generate or expand transport types and then deliberately choose presentation fields.

Useful fields stored in the imported source but not fully exposed by the API

The importer also retains fields that may be useful after quality review:

Building/property state

- recorded status;
- building-part type;
- construction year;
- building completed flag;
- construction phase;
- new-construction flag;
- actual use;
- room composition;
- number of rooms;
- position in building;
- floor.

Areas and outdoor space

- sold area;
- sold building-part area;
- sold usable area;
- registered building-part area;
- usable area;
- atrium flag;
- atrium area;
- outdoor parking spaces.

Address and identifiers

- settlement;
- street;
- house number and suffix;
- cadastral municipality;
- building number;
- building-part number.

Transaction composition

- contractual price assigned to a component;
- sold share;
- VAT inclusion/rate where interpretation is safe;
- whether the transaction contains multiple building parts or parcels.

Land

- parcel number;
- parcel area;
- sold share;
- land type;
- verified-buildable classification;
- component price;
- mixed-package warning.

Rental-specific attributes

- furnishing;
- rented-space type;
- microlocation;
- storefront;
- shopping-centre flag;
- rented area;
- usable rented area;
- construction year;
- position in building;
- room count;
- hospitality garden and area;
- operating costs included;
- rental term and duration;
- market-rent classification.

Codex must verify the current schema/API before adding any item.

Recommended public property panel

Show only populated, meaningful fields.

Core

- address;
- property type with human-readable description;
- contract date;
- total amount;
- amount per m^2;
- relevant area definition;
- construction year;
- room count;
- new-construction status;
- quality/representative status.

Optional when available

- usable area;
- sold area;
- floor;
- specific location in building;
- parking spaces;
- atrium and atrium area;
- building completion/construction phase;
- building age at contract date;
- cadastral/building reference in a technical section.

Land-specific

- total parcel area;
- number of parcels;
- buildability status;
- buildability-readiness category;
- component price where defensible;
- mixed-transaction warning;
- strict verified-buildable inclusion status.

Rental-specific

- monthly rent excluding VAT;
- rent per m^2;
- lease start/end;
- duration;
- operating costs included;
- furnished/unfurnished;
- market-rent status;
- rented-space type.

Derived values that may add real value

Only derive these in a documented backend/domain layer, not ad hoc in JSX:

- building age at transaction date;
- price difference from same-year geography median;
- percentile within the comparison cohort;
- difference from model prediction;
- interval-relative rating;
- price/rent difference from the same building's historical median;
- sale-to-rent relationship or gross yield, only where sale and rent cohorts are comparable and methodology supports it;
- data completeness/quality score;

- verified-buildable land status.

Every derived value must expose:

- calculation method;
- comparison period;
- cohort/scope;
- sample size;
- reliability or warning;
- methodology/rule version where applicable.

Codebook descriptions

The importer stores ETN codebooks in `codebook_values` with:

- codebook name;
- numeric value;
- Slovenian description.

Use these descriptions for fields such as property type, actual use, furnishing, rented-space type, microlocation, construction phase, marketability, and land type where a matching codebook exists.

Required approach:

- resolve codes to stable domain DTOs in the backend;
- return both code and description where useful;
- do not make the browser download the full raw codebook for each property;
- cache stable codebook/domain metadata;
- keep unknown codes visible to diagnostics but do not display unexplained raw numbers to ordinary users;
- test missing, deprecated, and unknown codebook values.

Privacy and data-minimisation rules

Before exposing source fields, review them for personal or sensitive content.

Do not publicly display without an explicit decision:

- free-text transaction notes;
- free-text property notes;
- internal labels;
- apartment/business unit number where it could identify a private household;
- raw reviewer identity;
- any personal information accidentally contained in source notes.

Prefer a whitelist of approved public fields, not a blacklist.

---

19. Display-value tests

Add unit/component tests proving that:

1. a missing floor produces no `Nadstropje` row;
2. a blank floor produces no row;
3. a meaningful floor such as `3` or `P` is shown only in the detailed panel;
4. generic `positionInBuilding = "nadstropje"` is omitted from the compact popup;
5. a specific meaningful position may appear in the detailed panel;
6. missing price is not rendered as a fake zero;
7. missing optional fields do not leave empty separators;
8. zero is retained where zero is a valid value;
9. codebook-backed values use Slovenian descriptions rather than raw codes;
10. multiple parts/parcels are presented as a composition, not by silently choosing the first row;

11. property notes/internal identifiers are not leaked;
12. the screen reader output does not announce hidden/missing rows.

Add an end-to-end regression fixture matching the reported example:

```
{
  "usableArea": 29,
  "floor": null,
  "positionInBuilding": "nadstropje"
}
```

Expected compact UI:

`Uporabna površina: 29 m²`

It must not contain:

- `Nadstropje`;
- `Ni podatka`;
- `Položaj`;
- `nadstropje`.

---

20. Additional definition of done

- Compact property popups no longer show rows with unavailable data.
- `Nadstropje: Ni podatka` is removed.
- Generic `Položaj: nadstropje` is removed from the compact view.
- The detailed panel uses meaningful-value rules consistently.
- Codex documents that `positionInBuilding` is actual ETN source data, not a hardcoded frontend value.
- A field-availability audit identifies which additional real-estate attributes are worth showing.
- Useful API fields are no longer discarded by incomplete frontend types.
- ETN numeric codes are displayed through human-readable domain descriptions.
- Public exposure follows a field whitelist and does not leak raw notes or sensitive unit identifiers.